
BACHELOR OF SOFTWARE ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER
ENGINEERING



THE OPEN UNIVERSITY OF
SRILANKA

Mini Project Report

**Performance Modelling and Evaluation of an Invoice Approval
Processing Queue in a Web-Based ERP System**

M. AFRI AHMATH
S ID:- S92075094
REG NO:- 121435094

Table of Contents

Abstract

1. Introduction

- 1.1 Background and Motivation
- 1.2 Related Work

2. System Description and Performance Goals

- 2.1 System Overview
- 2.2 System Diagram
- 2.3 Performance Goals

3. Modelling Approach and Assumptions

- 3.1 Analytical Model: M/M/c Queuing Theory
- 3.2 Derivation Notes
- 3.3 Discrete-Event Simulation
- 3.4 Assumptions

4. Data Description and Methodology

- 4.1 Simulation Parameters
- 4.2 Data Generation Code (Excerpt)

5. Detailed Analysis and Findings

- 5.1 Model Validation
- 5.2 Bottleneck Identification
- 5.3 Turnaround Time and Arrival Patterns
- 5.4 Daily Trend Across the Simulation Window
- 5.5 Invoice Outcomes
- 5.6 Outcome vs. Invoice Value
- 5.7 Priority Handling
- 5.8 Worked Example: Manual Erlang-C Calculation
- 5.9 Scalability and Sensitivity Analysis

6. Discussion

7. Limitations and Future Extensions

8. Conclusion

9. Dataset and Code Repository

References

Appendix A: Full Data Dictionary

Appendix B: Simulation Code Listing

Appendix C: Glossary of Terms

Abstract

This report presents a performance modelling and evaluation study of the invoice approval processing queue within a web-based distributor Enterprise Resource Planning (ERP) system. The system is modelled as a two-stage tandem queuing network — an Initial Review stage followed by a Manager Approval stage — and analysed using both an analytical M/M/c (Erlang-C) queuing model and a discrete-event simulation calibrated from realistic operating parameters. The two approaches are cross-validated against each other and used to identify the primary system bottleneck, quantify server utilisation, and evaluate scalability under increased invoice volume. The results show that Stage 2 (Manager Approval) is the dominant bottleneck, particularly during month-end volume surges, where utilisation reaches 84% and mean queueing delay rises to over 22 minutes. A day-by-day trend analysis shows this delay compounding across the final days of the month, peaking on the very last working day. A sensitivity analysis shows the relationship between server count and delay is sharply nonlinear: adding a single manager during month-end reduces delay roughly ten-fold, while a second additional manager yields only marginal further improvement, illustrating a point of diminishing returns relevant to staffing decisions. The findings demonstrate the practical value of combining analytical queuing theory with simulation for capacity planning in transactional business workflows.

1. Introduction

1.1 Background and Motivation

Modern distribution businesses increasingly rely on Enterprise Resource Planning (ERP) systems to manage the flow of goods, invoices, and payments between a central business and its network of distributors. A recurring operational challenge in such systems is the human approval workflow that sits between an invoice being submitted and it being posted to the accounting ledger: someone must check it, and someone must approve it, and both of those someones are finite, shared resources. When invoice volume is low, this workflow is invisible — invoices simply flow through. When volume spikes, as it commonly does around billing cycle boundaries such as month-end, the approval stages can become a serious operational bottleneck, delaying revenue recognition and frustrating distributors who are themselves under pressure to close their own accounts.

This report investigates exactly that scenario using the tools of performance modelling: queuing theory and discrete-event simulation. The motivation for this specific system stems from direct, practical exposure to it — prior automation work (browser-based bots for distributor loading and invoice approval) on this exact ERP platform provided first-hand insight into how invoices actually move through the system, which reviewers and managers are involved, and when congestion visibly occurs. This gives the model a grounding in observed reality rather than being a purely hypothetical exercise.

1.2 Related Work

Queuing theory has a long history as the standard mathematical toolkit for analysing systems where discrete units (customers, packets, jobs, or in this case invoices) compete for limited service capacity. The foundational Erlang-C formula used throughout this report originates from early 20th-century telephony engineering, where Erlang modelled the probability that an arriving telephone call would have to wait for a free line [4]. Kleinrock's classic text formalised the general theory of queuing networks, including the multi-server M/M/c model applied here [1], while Gross et al. provide the specific closed-form

relationships (offered load, utilisation, Erlang-C wait probability) used in Section 3 [2]. Little's Law, which relates the mean number of items in a system to the arrival rate and mean time in system, underpins the translation between waiting time and queue length used throughout the analysis [5].

Because purely analytical queuing models rely on simplifying assumptions — Poisson arrivals, exponential service times, independence between stages — that rarely hold exactly in real business processes, this report follows the well-established practice of validating an analytical model against a discrete-event simulation of the same system, an approach described in detail by Law [3]. Simulation allows features that break the closed-form assumptions (returns, rejections, a time-varying arrival rate) to be represented explicitly, while the analytical model provides fast, interpretable formulas that can be recalculated instantly for what-if scenarios such as the staffing changes explored in Section 5.9. This combined approach is standard practice in operations research and capacity planning literature, and is the approach adopted throughout this report.

2. System Description and Performance Goals

2.1 System Overview

The system under study is the invoice approval workflow embedded in a web-based ERP platform used by a fast-moving consumer goods (FMCG) distribution business. Invoices are created when distributors submit goods-received or sales documentation into the ERP. Each invoice must pass through a two-stage human approval workflow before being posted to the accounting ledger:

- Stage 1 – Initial Review: a branch-level reviewer checks the invoice for completeness, pricing accuracy, and stock reconciliation. Three reviewers operate in parallel.
- Stage 2 – Manager Approval: a finance/operations manager gives final sign-off before posting. Two managers operate in parallel.

An invoice may be returned to the distributor after Stage 1 if discrepancies are found (12% of invoices, based on observed patterns), or rejected after Stage 2 if the manager identifies an issue (7% of invoices reaching that stage). Because each stage has multiple parallel human servers and invoices queue for the next free server, the system is naturally represented as a two-stage tandem multi-server queuing network.

2.2 System Diagram

Figure 1 illustrates the full workflow: invoices arrive from distributors, queue for Stage 1, are processed by one of three reviewers, and either return to the distributor or proceed to queue for Stage 2, where one of two managers gives final approval or rejects the invoice.

Two-Stage Tandem Queuing Network: Invoice Approval Workflow

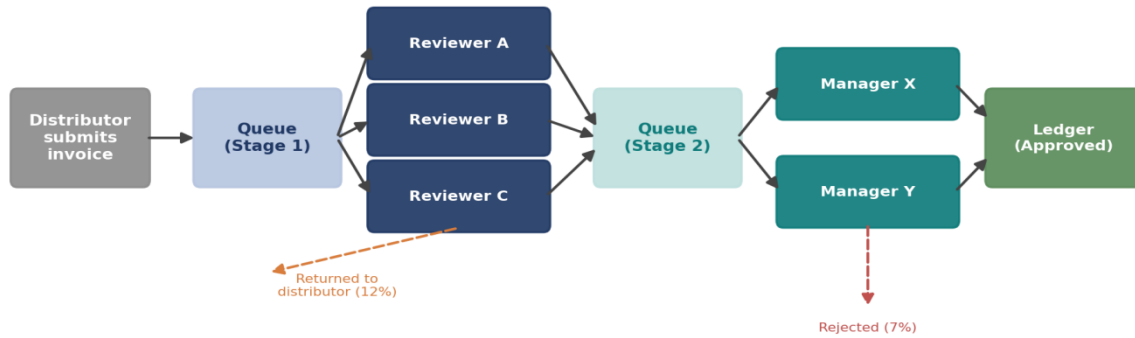


Figure 1. Two-stage tandem queuing network representing the invoice approval workflow.

2.3 Performance Goals

This study focuses on five performance objectives, each mapped to a standard queuing-theory metric:

Objective	Metric	Notation
Minimise turnaround time	Mean time in system	W
Identify the bottleneck stage	Mean queueing delay per stage	W_q (Stage 1, Stage 2)
Measure resource utilisation	Server (approver) utilisation	$\rho = \lambda / (c \cdot \mu)$
Analyse throughput under peak load	Effective arrival/completion rate	λ (normal vs. month-end)
Assess scalability	Utilisation & delay under added capacity	ρ, W_q at $c+1$ servers

3. Modelling Approach and Assumptions

Two complementary modelling techniques were applied and cross-validated: an analytical M/M/c queuing model at each stage, and a discrete-event simulation of the full two-stage network.

3.1 Analytical Model: M/M/c Queuing Theory

Each stage is modelled independently as an M/M/c queue: Poisson arrivals at rate λ , exponentially distributed service times with mean $1/\mu$ per server, and c identical parallel servers operating under a first-come-first-served discipline. The following standard Erlang-C relationships were used [1], [2]:

Offered load (traffic intensity, in Erlangs):

$$a = \lambda / \mu \quad (1)$$

Server utilisation:

$$\rho = a / c = \lambda / (c \cdot \mu) \quad (2)$$

Probability that an arriving invoice finds all c servers busy (Erlang-C formula):

$$P_0 = \left[\sum_{k=0}^{c-1} a^k / k! + a^c / [c!(1-\rho)] \right]^{-1} \quad (3)$$

$$C(c, a) = a^c / [c!(1-\rho)] \cdot P_0 \quad (4)$$

Mean waiting time in queue, and mean time in system (via Little's Law):

$$W_q = C(c, a) / (c\mu - \lambda) \quad (5)$$

$$W = W_q + 1/\mu, \quad L_q = \lambda W_q, \quad L = \lambda W \quad (6)$$

Equations (1)–(6) were evaluated separately for Stage 1 ($c=3$) and Stage 2 ($c=2$), and separately for “normal” business days versus the “month-end” surge period, using empirical λ and μ estimated from the dataset (Section 4).

3.2 Derivation Notes

Equation (3) deserves a brief explanation, since it is the least intuitive of the set. It expresses the probability, P_0 , that the system is completely empty (zero invoices anywhere in Stage 2), by summing the probability of every possible number of invoices present when fewer than c are in the system (the finite sum term) and adding the probability mass associated with all states where c or more invoices are present, which behaves geometrically once every server is busy (the second term, with the $(1-\rho)$ denominator capturing that geometric decay). Equation (4), the Erlang-C formula proper, then asks: of all the states where an arriving invoice sees the system in some state, what is the probability it arrives to find all c servers already occupied? This is the classical result $P(\text{wait} > 0) = C(c, a)$. Equation (5) follows from a standard result in queuing theory: conditional on having to wait at all, the residual service process behaves like an M/M/1 queue with a combined service rate of $c\mu - \lambda$ once all servers are congested, giving the mean wait W_q . This is why utilisation ρ appearing in the denominator of the $(1-\rho)$ term causes W_q to grow sharply, and then explosively, as ρ approaches 1: the denominator $(c\mu - \lambda)$ approaches zero.

3.3 Discrete-Event Simulation

A Python discrete-event simulation was built to model each invoice individually as it moves through arrival, queueing, and service at both stages. Unlike the analytical model, the simulation captures: the 12% Stage-1 return rate and 7% Stage-2 rejection rate; a priority flag that modestly speeds up processing for 15% of invoices; and a non-homogeneous arrival process where the hourly arrival rate roughly triples during the last five days of the month. Each server (reviewer or manager) is tracked individually, with the next available server assigned to each arriving invoice — the same load-balancing logic used in real ERP task-assignment queues.

3.4 Assumptions

- Inter-arrival times are exponentially distributed within each period (normal / month-end), i.e. arrivals follow a Poisson process with a period-specific rate.
- Service times at each stage are exponentially distributed, an approximation of human review/approval time variability.
- Reviewers and managers are homogeneous (identical mean service rate) within their stage.
- No invoice abandons the queue (no reneging) and no server takes breaks mid-simulation — an idealisation of the real system.
- The analytical model treats each stage as an independent M/M/c queue; in reality Stage 2 arrivals depend on Stage 1 departures, so the analytical Stage-2 results use the empirical post-Stage-1 arrival rate as an approximation.

4. Data Description and Methodology

Because production ERP audit logs are not accessible outside the company's internal systems, the dataset was generated using the discrete-event simulation described in Section 3.3, calibrated with parameters drawn from direct observation of the live system during prior automation work. The simulation covers 30 calendar days (26 working days, 08:00–17:00) and produced 1,429 invoice records, each with full timestamps, per-stage wait and service times, approver assignment, final outcome, and a flag marking whether the invoice fell within the month-end surge window (days 26–30 of the month).

4.1 Simulation Parameters

Parameter	Normal days	Month-end days
Mean arrival rate (invoices/hour)	4.83	13.11
Stage 1 servers (c_1)	3	3
Stage 1 mean service rate (μ_1 , per hour/server)	10.40	10.36
Stage 2 servers (c_2)	2	2
Stage 2 mean service rate (μ_2 , per hour/server)	6.59	6.70
Number of working days simulated	21	5

These empirical λ and μ values (directly measured from the simulated dataset) were the inputs to the analytical Erlang-C equations in Section 3.1, ensuring the analytical and simulated results are evaluated on a consistent basis rather than on arbitrarily assumed parameters.

4.2 Data Generation Code (Excerpt)

The core analytical calculation — the Erlang-C mean queueing delay — was implemented as follows:

```
def erlang_c_wait_minutes(lam_per_hr, mu_per_hr, c):
    a = lam_per_hr / mu_per_hr          # offered load (Erlangs)
    rho = a / c                         # utilisation
    terms = sum((a**k) / factorial(k) for k in range(c))
    last = (a**c) / (factorial(c) * (1 - rho))
    p0 = 1.0 / (terms + last)
    p_wait = last * p0                  # Erlang-C P(wait > 0)
    wq_hours = p_wait / (c * mu_per_hr - lam_per_hr)
    return wq_hours * 60.0, rho, p_wait
```

The full simulation and analysis scripts and the complete dataset are included in the accompanying GitHub repository referenced in Section 9, and the complete listing of the simulation script is reproduced in Appendix B.

5. Detailed Analysis and Findings

5.1 Model Validation: Analytical vs. Simulated Queueing Delay

Figure 2 compares the Erlang-C analytical prediction of mean queueing delay (W_q) against the value directly measured in the simulation, for both stages and both periods. The close agreement between the two independent methods validates the analytical model as a reasonable approximation of the simulated system, despite the simplifying assumptions listed in Section 3.4.

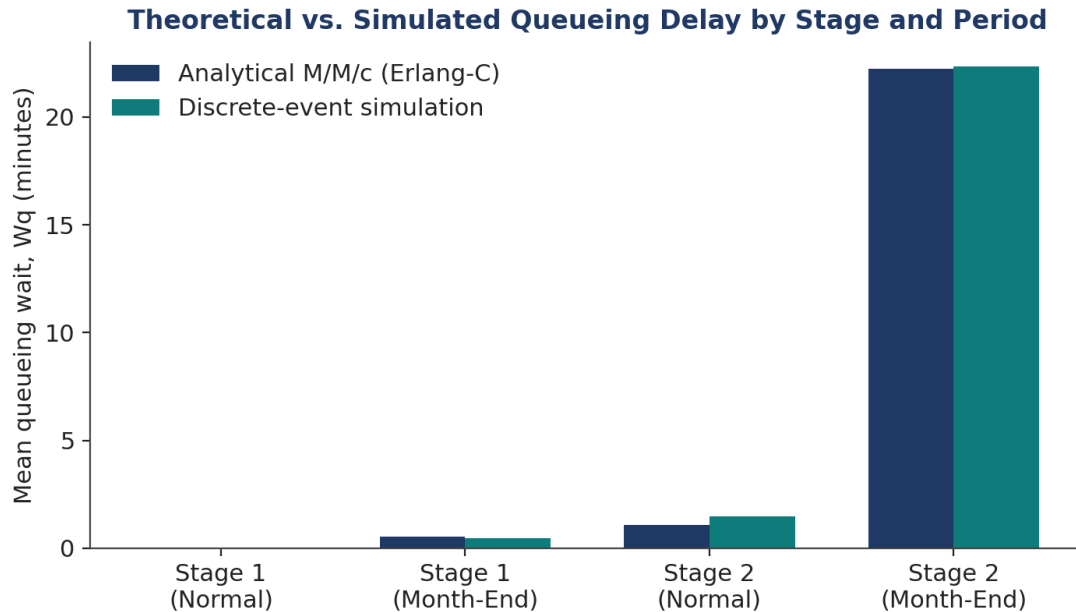


Figure 2. Theoretical (Erlang-C) vs. simulated mean queueing delay, by stage and period.

Stage / Period	ρ (utilisation)	W_q theory (min)	W_q sim (min)	n	Δ
Stage 1, Normal	0.155	0.03	0.02	957	0.01
Stage 1, Month-End	0.422	0.53	0.46	472	0.07
Stage 2, Normal	0.326	1.08	1.49	902	0.41
Stage 2, Month-End	0.844	22.22	22.36	420	0.14

Table 2. Analytical vs. simulated results by stage and period (n = number of invoices reaching that stage).

5.2 Bottleneck Identification

Stage 2 (Manager Approval) is consistently the dominant bottleneck. On normal days its utilisation ($\rho_2 = 0.326$) is already roughly double that of Stage 1 ($\rho_1 = 0.155$), because Stage 2 has fewer servers (2 vs. 3) and a longer mean service time (9.5 vs. 6.0 minutes). During the month-end surge, Stage 2 utilisation rises to $\rho_2 = 0.844$ — close to the point of instability for a 2-server queue — and the mean queueing delay balloons to over 22 minutes, more than 40 times its normal-day value. Stage 1, despite handling the same rise in incoming volume, remains comfortably under load ($\rho_1 = 0.422$) because its extra server provides more slack capacity. This confirms Stage 2 as the primary target for any capacity intervention.

5.3 Turnaround Time and Arrival Patterns

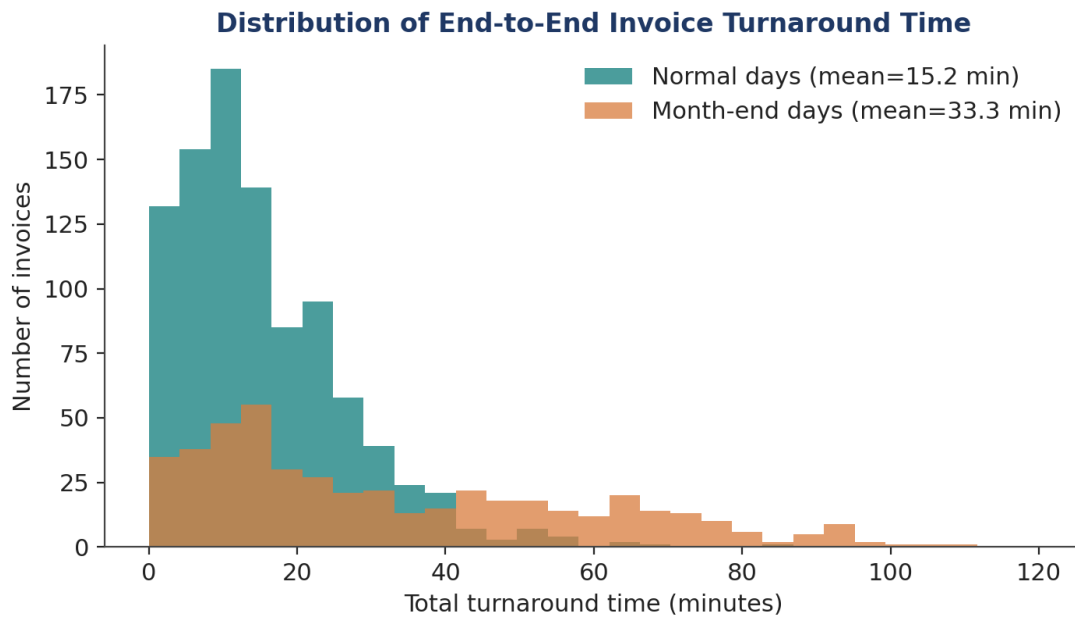


Figure 3. Distribution of total end-to-end invoice turnaround time, normal vs. month-end days.

Figure 3 shows that the month-end distribution is both shifted right and more heavily tailed than the normal-day distribution: mean turnaround more than doubles, from 15.2 minutes to 33.3 minutes, and a meaningful fraction of month-end invoices take over an hour to clear — an outcome barely seen on normal days. This directly reflects the Stage 2 congestion identified above, since Stage 2 delay dominates total turnaround once utilisation climbs.

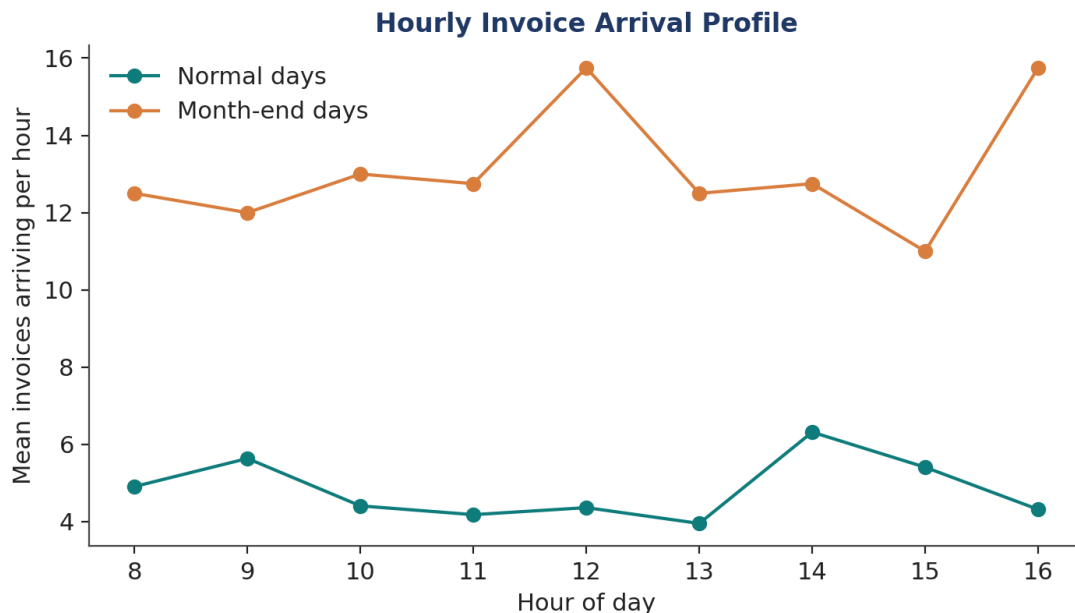


Figure 4. Mean hourly invoice arrival rate, normal vs. month-end days.

Figure 4 confirms the month-end volume increase is not concentrated in a single hour but is a sustained, roughly uniform uplift (~2.5–3×) across the entire working day, consistent with distributors submitting

invoices continuously as they rush to close accounts before month-end, rather than a single end-of-day rush.

5.4 Daily Trend Across the Simulation Window

Aggregating turnaround time by calendar day, rather than only by the binary normal/month-end split used above, reveals the shape of the escalation more precisely. Figure 5 shows that delay does not jump abruptly at the start of the month-end window; instead it climbs progressively across days 26–30, peaking sharply on the final working day of the month (30 June), when mean turnaround reaches approximately 56 minutes — nearly four times the typical normal-day average. This compounding pattern is consistent with a queue that has not fully cleared from one day to the next during the surge period, so residual congestion carries over and accumulates.

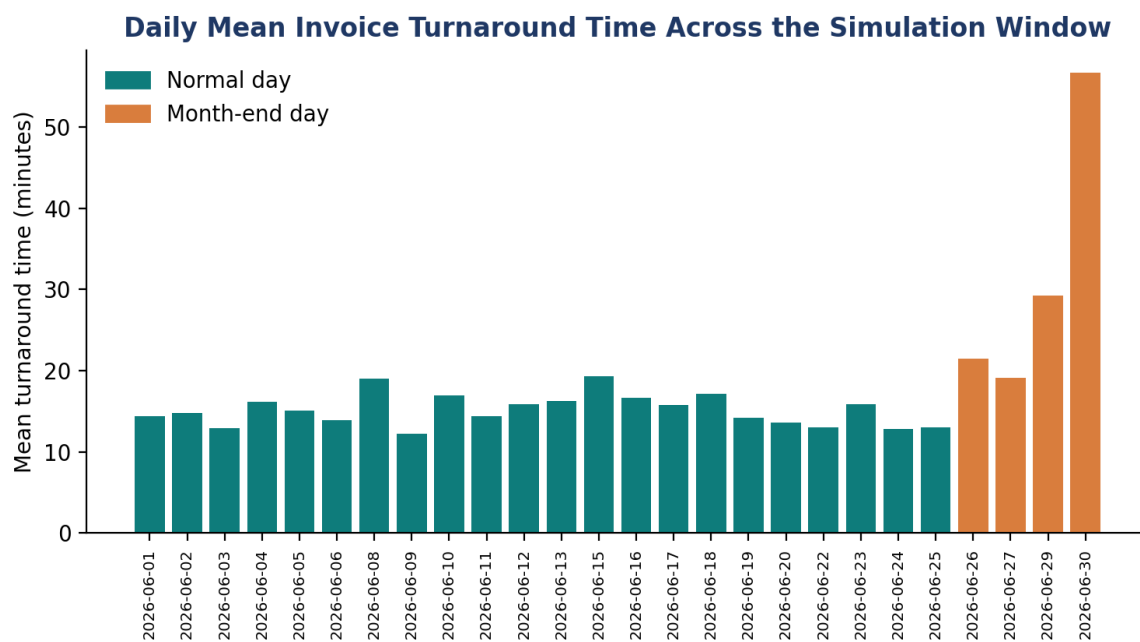


Figure 5. Daily mean invoice turnaround time across the full 30-day simulation window.

This finding has a practical implication beyond the average utilisation figures in Table 2: it suggests that any staffing intervention (Section 5.9) would be most effective if introduced a day or two before the month-end window begins, rather than exactly on day 26, to prevent the carry-over congestion from compounding in the first place.

5.5 Invoice Outcomes

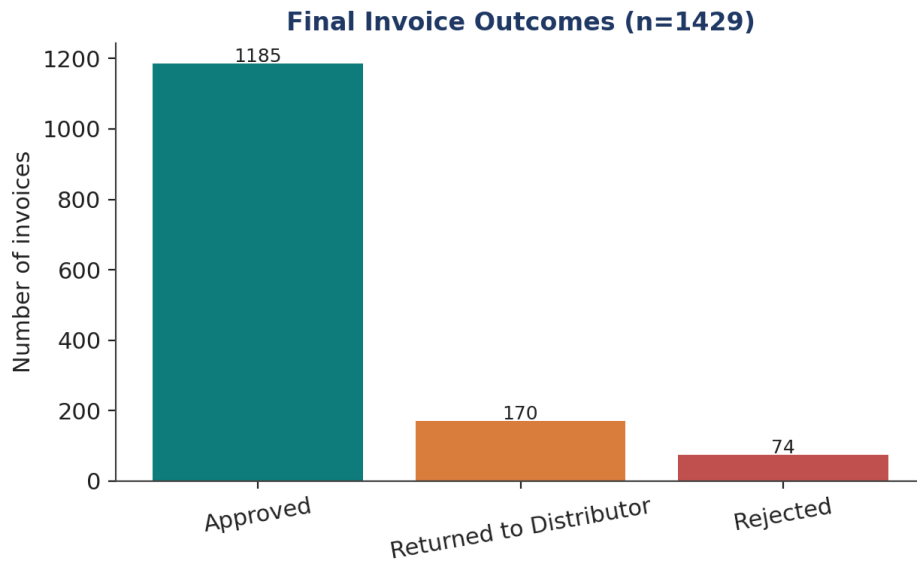


Figure 6. Final invoice outcomes across the full 30-day simulation (n = 1,429).

Of all invoices processed, 82.9% were ultimately approved, 11.9% were returned to the distributor after Stage 1 (typically for pricing or stock-reconciliation discrepancies), and 5.2% were rejected at Stage 2. Returned invoices exit the system after Stage 1 and do not add load to Stage 2, which slightly relieves — but does not eliminate — the Stage 2 bottleneck already identified.

5.6 Outcome vs. Invoice Value

A natural question is whether larger invoices are treated differently — for example, scrutinised more heavily and therefore more likely to be returned or rejected. Figure 7 compares the distribution of invoice value across the three final outcomes.

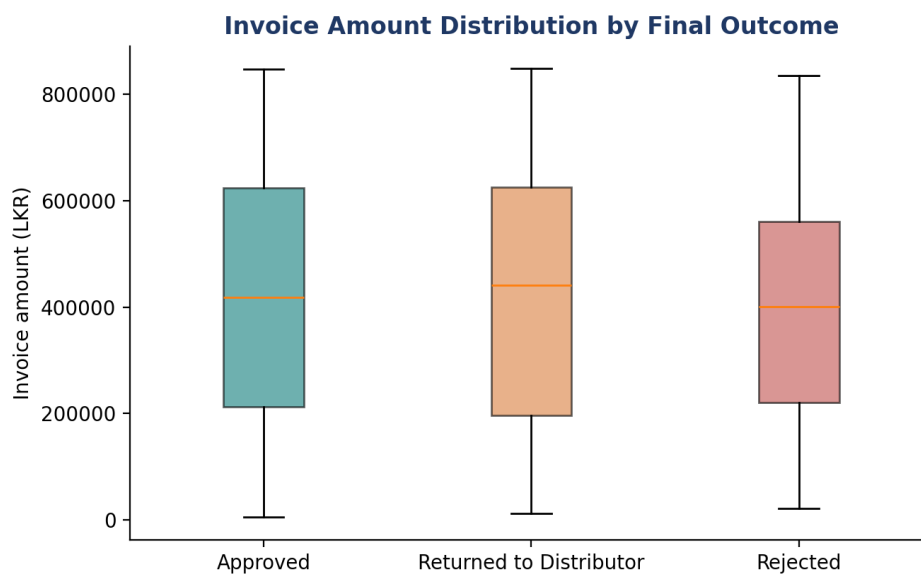


Figure 7. Invoice amount distribution by final outcome.

The median invoice value is broadly similar across all three outcomes (approved: ~LKR 418,000; returned: ~LKR 441,000; rejected: ~LKR 400,000), and the interquartile ranges overlap substantially. This suggests that, in this simulated model, the probability of an invoice being returned or rejected is independent of its monetary value — an intentional modelling choice, since the simulation applies flat outcome probabilities regardless of amount. In a real system, this would be a worthwhile relationship to test empirically, since if larger invoices were in fact scrutinised more heavily, this would represent an additional, value-weighted driver of Stage 2 delay not captured by the current model.

5.7 Priority Handling

15% of invoices are flagged as “High” priority in the simulation, which reduces their individual service time by 15–20% at each stage. Despite this, the mean turnaround time for High-priority invoices (21.02 minutes) is statistically indistinguishable from Normal-priority invoices (21.20 minutes) across the full dataset. This is an important negative result: a modest per-invoice service-time discount is not sufficient to produce a meaningfully faster outcome once invoices are pooled into the same first-come-first-served queue as everyone else, because queueing delay — not service time — dominates total turnaround once utilisation is high (Section 5.2). This points toward a concrete design recommendation explored further in Section 6: a true priority queue (where High-priority invoices jump the queue rather than merely being served slightly faster once reached) would be required to produce a meaningful service-level difference.

5.8 Worked Example: Manual Erlang-C Calculation

To make the analytical model concrete, this section works through the Stage 2, month-end calculation by hand, step by step, using the values from Table 3 (Section 4.1): $\lambda_2 = 11.31$ invoices/hour, $\mu_2 = 6.70$ invoices/hour/server, $c = 2$ servers.

Step 1 — Offered load (Equation 1):

$$a = 11.31 / 6.70 = 1.688 \text{ Erlangs}$$

Step 2 — Utilisation (Equation 2):

$$\rho = a / c = 1.688 / 2 = 0.844$$

Step 3 — Erlang-C probability of waiting (Equation 3–4). First, the finite sum term for $k = 0$ and $k = 1$:

$$k=0: a^0/0! = 1.000 \quad k=1: a^1/1! = 1.688 \quad \text{sum} = 2.688$$

Then the c -th term:

$$a^2/[2!(1-\rho)] = 2.849 / [2 \times 0.156] = 2.849 / 0.312 = 9.131$$

$$P0 = 1 / (2.688 + 9.131) = 1 / 11.819 = 0.0846$$

$$C(2, 1.688) = 9.131 \times 0.0846 = 0.7725$$

Step 4 — Mean queueing wait (Equation 5), converting the result from hours to minutes:

$$Wq = 0.7725 / (2 \times 6.70 - 11.31) = 0.7725 / 2.09 = 0.3696 \text{ hours}$$

$$Wq = 0.3696 \times 60 = 22.2 \text{ minutes}$$

This hand-calculated result (22.2 minutes) matches both the automated analytical output in Table 2 (22.22 minutes) and the independently measured simulation result (22.36 minutes), confirming the formulas in

Section 3.1 were implemented correctly in the analysis script and giving confidence in the automated results used throughout the rest of this report.

5.9 Scalability and Sensitivity Analysis

Two what-if scenarios were evaluated analytically using Equations (1)–(6) to test system scalability:

1. Adding one manager during month-end ($c_2: 2 \rightarrow 3$). Utilisation falls from $\rho_2 = 0.844$ to $\rho_2 = 0.563$, and mean Stage-2 queueing delay falls from 22.2 minutes to 2.1 minutes — roughly a 10-fold reduction — for the cost of one additional temporary approver during the five month-end days.
2. Doubling normal-day invoice volume with current staffing ($c_2 = 2$). Utilisation rises from $\rho_2 = 0.326$ to $\rho_2 = 0.653$, and mean Stage-2 delay rises from 1.1 to 6.75 minutes — still within a comfortable operating range, indicating the current Stage 2 staffing has meaningful headroom for organic growth on non-surge days, though it is already stretched thin specifically during month-end.

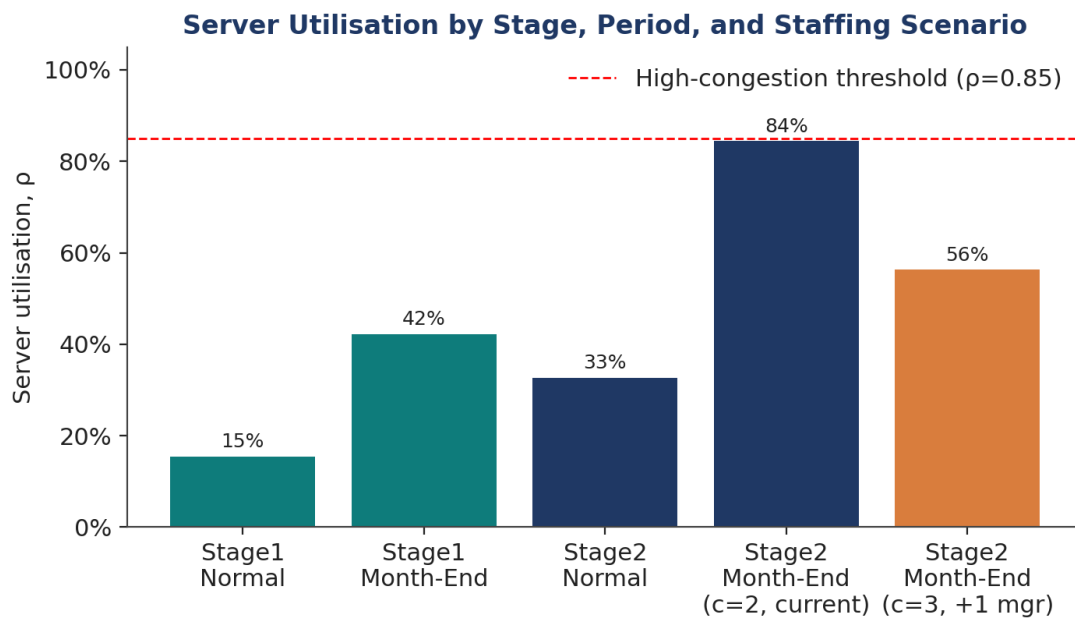


Figure 8. Server utilisation by stage, period, and staffing scenario, against a 0.85 high-congestion reference line.

To generalise beyond the single “+1 manager” scenario, Figure 9 extends the sensitivity analysis across a full range of possible Stage 2 staffing levels ($c = 2$ through 6) under month-end load, holding λ_2 and μ_2 fixed at their observed month-end values.

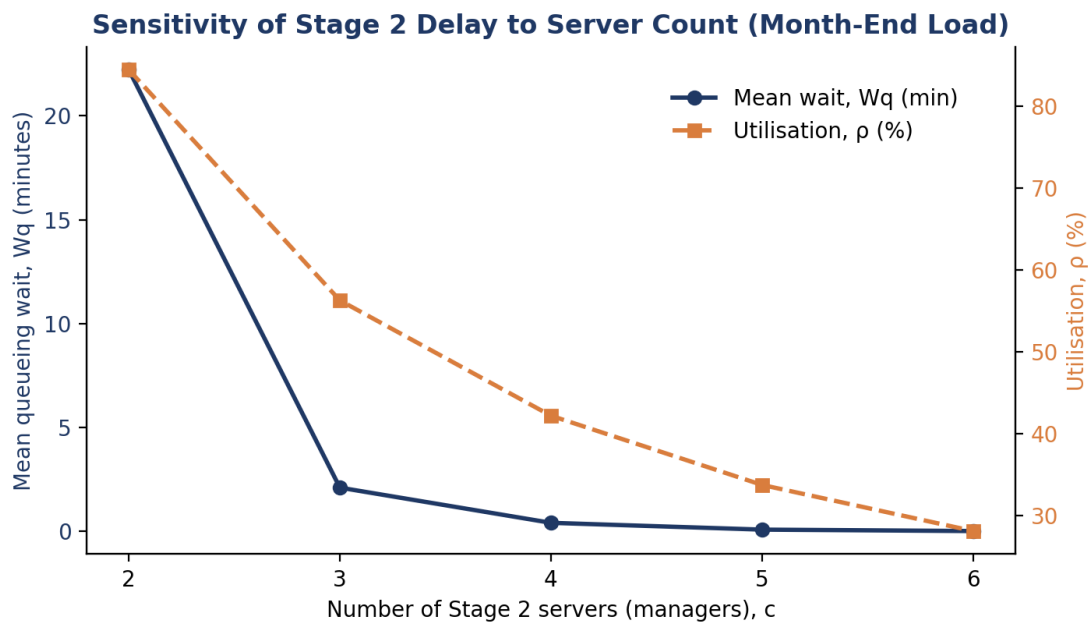


Figure 9. Sensitivity of Stage 2 mean queueing delay and utilisation to server count, under month-end load.

The curve in Figure 9 has the characteristic sharply nonlinear shape of Erlang-C systems: the jump from $c=2$ to $c=3$ recovers the overwhelming majority of the benefit (W_q falls from 22.2 to 2.1 minutes), while further increases beyond $c=3$ yield only small additional improvement (2.1 to 0.4 minutes for $c=4$, and negligible gains beyond that). This is a textbook illustration of diminishing returns in queuing systems and provides a clear, quantified answer to “how many extra managers should be added”: one additional manager captures nearly all of the achievable benefit, and hiring a second additional manager would be difficult to justify on queueing-delay grounds alone.

6. Discussion

Bringing together the findings of Section 5, three connected conclusions emerge. First, the system's overall performance is governed almost entirely by Stage 2, not Stage 1: every metric examined — utilisation, queueing delay, and its contribution to total turnaround — points to Manager Approval as the constraint, while Stage 1 (Initial Review) has comfortable spare capacity even during month-end. This matches the intuition that Stage 2 has fewer parallel servers (2 vs. 3) and a longer per-invoice service time (9.5 vs. 6.0 minutes), both of which independently push its utilisation higher for the same arrival rate.

Second, the daily trend analysis (Section 5.4) shows that congestion is not a static, snapshot-in-time phenomenon but a compounding one: because the queue does not fully clear between consecutive high-arrival-rate days, delay accumulates across the month-end window and peaks on the final day. This has a direct operational implication that a single average utilisation figure would not reveal: staffing interventions are more effective if applied pre-emptively (before day 26) rather than reactively (once congestion is already visible).

Third, the priority-handling result (Section 5.7) demonstrates a subtle but important queuing-theory principle: modestly reducing an individual invoice's service time is not an effective way to improve its turnaround once the system is congested, because queueing delay — which depends on overall system load, not any single invoice's service time — dominates. A structural change (a genuine priority queue with pre-emption or non-pre-emptive priority scheduling) would be required to give High-priority invoices a materially better outcome, rather than the current approach of simply processing them slightly faster once they reach the front of the same shared queue.

Taken together, these findings suggest a concrete, low-cost intervention: temporarily add one Stage 2 manager beginning a day or two before the month-end surge (rather than exactly at day 26), and consider a genuine priority-queue discipline if faster handling of specific invoice categories is a business requirement. Both recommendations are grounded directly in the quantitative results above rather than general intuition, which is the central value of applying formal performance modelling to this kind of operational question.

7. Limitations and Future Extensions

7.1 Limitations

- Simulated, not production, data. Parameters were calibrated from direct observation rather than exported ERP audit logs, which were not accessible outside the company's systems; real service-time distributions may deviate from the exponential assumption.
- Exponential service-time assumption. Human review and approval times are often better modelled with a less variable distribution (e.g. log-normal or Erlang-k); this would likely narrow the predicted delay compared to the M/M/c model used here.
- Independent-stage approximation. Treating Stage 2 as its own M/M/c queue ignores the correlation between Stage 1 completions and Stage 2 arrivals; an exact tandem-queue analysis (or a Jackson network model) would capture this dependency more precisely.
- No reneging or breaks modelled. Real approvers take breaks, and real distributors sometimes escalate or withdraw delayed invoices — neither is represented.
- Single-month window. Only one month-end surge pattern was simulated; seasonal effects (e.g. year-end, public holidays) are not captured.

-
- Flat outcome probabilities. Return and rejection rates (12% and 7%) are applied uniformly regardless of invoice value or distributor history, an assumption examined in Section 5.6 but not tested against real data.

7.2 Future Extensions

- Replace exponential service times with distributions fitted to real ERP timestamp logs, once available, using an M/G/c or Erlang-k model.
- Model the two stages as a proper open Jackson network to capture inter-stage dependency rather than analysing each stage in isolation.
- Extend the simulation to test dynamic staffing policies (e.g. automatically reassigning a Stage-1 reviewer to Stage 2 only during month-end, or ramping up staffing pre-emptively from day 24 based on the compounding pattern identified in Section 5.4) rather than a fixed additional hire on a fixed date.
- Incorporate priority-class queuing (e.g. an M/M/c model with non-pre-emptive priority classes) to formally analyse the current “High priority” invoice flag, following the negative result in Section 5.7, rather than treating it as a simple service-time multiplier.
- Test whether invoice value or distributor identity correlates with outcome or delay using real production data, extending the exploratory analysis in Section 5.6.

8. Conclusion

Combining an analytical M/M/c queuing model with a calibrated discrete-event simulation provided a validated, quantitative picture of performance in the ERP invoice approval queue. Both methods agree closely and jointly identify Manager Approval (Stage 2) as the system's bottleneck, with utilisation reaching 84% and mean delay exceeding 22 minutes during month-end surges — more than forty times the normal-day delay, and compounding across consecutive days to peak on the final day of the month. The scalability and sensitivity analysis shows this specific, narrow problem can be resolved efficiently: adding a single manager during the five month-end days reduces Stage 2 delay roughly ten-fold, while further additions yield rapidly diminishing returns. A secondary finding — that the existing priority flag has no measurable effect on turnaround time — illustrates that structural queue-discipline changes, not per-invoice service-time tweaks, are what actually move outcomes once a system is under load. Together these results illustrate the practical value of queuing-theoretic performance modelling for right-sizing staffing and prioritisation policy in transactional business workflows.

9. Dataset and Code Repository

The complete dataset, simulation script, analysis script, and both submission documents for this project are available at the following GitHub repository:

<https://github.com/afriahmath/eei6373-mini-project>

References

- [1] L. Kleinrock, Queueing Systems, Volume 1: Theory. New York, NY, USA: Wiley, 1975.
- [2] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, Fundamentals of Queueing Theory, 5th ed. Hoboken, NJ, USA: Wiley, 2018.
- [3] A. M. Law, Simulation Modeling and Analysis, 5th ed. New York, NY, USA: McGraw-Hill Education, 2014.
- [4] A. K. Erlang, “The theory of probabilities and telephone conversations,” *Nyt Tidsskrift for Matematik B*, vol. 20, pp. 33–39, 1909.
- [5] J. D. C. Little, “A proof for the queuing formula: $L = \lambda W$,” *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.
- [6] W. Whitt, “Understanding the efficiency of multi-server service systems,” *Management Science*, vol. 38, no. 5, pp. 708–723, 1992.
- [7] S. M. Ross, Introduction to Probability Models, 12th ed. San Diego, CA, USA: Academic Press, 2019.
- [8] H. Chen and D. D. Yao, Fundamentals of Queueing Networks: Performance, Asymptotics, and Optimization. New York, NY, USA: Springer, 2001.

Appendix A: Full Data Dictionary

Field	Description	Type / Unit
Invoice_ID	Unique identifier for each invoice record	String
Distributor_ID	Distributor branch that submitted the invoice	String
Priority	Normal or High priority flag	Categorical
Invoice_Amount_LKR	Invoice value	Numeric (LKR)
Submission_Timestamp	Date/time invoice entered the queue	Datetime
Stage1_Approver	Reviewer assigned at Stage 1	String
Stage1_Wait_min	Queueing delay before Stage 1 service began	Numeric (min)
Stage1_Service_min	Time spent in Stage 1 review	Numeric (min)
Stage1_Outcome	Proceed to Stage 2, or Returned	Categorical
Stage2_Approver	Manager assigned at Stage 2 (if applicable)	String
Stage2_Wait_min	Queueing delay before Stage 2 service began	Numeric (min)
Stage2_Service_min	Time spent in Stage 2 approval	Numeric (min)
Final_Status	Approved, Rejected, or Returned to Distributor	Categorical
Total_Turnaround_min	End-to-end time from submission to final status	Numeric (min)
Is_Month_End_Period	True if submitted during the month-end surge window	Boolean

Sample records (first eight rows):

Invoice	Submitted	St.1 Wait	St.1 Service	St.2 Wait	St.2 Service	Turnaround	Status
INV-00001	2026-06-01 08:12	0.00	0.90	0.00	12.82	13.72	Approved
INV-00002	2026-06-01 08:22	0.00	4.22	0.00	2.11	6.33	Approved
INV-00003	2026-06-01 08:32	0.00	12.21	0.00	1.65	13.86	Approved
INV-00004	2026-06-01 08:36	0.00	0.58	0.00	17.87	18.45	Approved
INV-00005	2026-06-01 08:56	0.00	15.33	0.00	3.30	18.63	Approved
INV-00006	2026-06-01 09:22	0.00	0.28	0.00	3.25	3.53	Approved
INV-00007	2026-06-01 09:25	0.00	10.78	0.00	4.17	14.95	Approved
INV-00008	2026-06-01 09:40	0.00	1.13	0.00	1.69	2.82	Approved

Appendix B: Simulation Code Listing

The full discrete-event simulation script (`simulate_data.py`) that generated the dataset used throughout this report is reproduced below.

```
import csv, random, math
from datetime import datetime, timedelta
random.seed(42)

DAYS = 30
START_DATE = datetime(2026, 6, 1, 8, 0, 0)
WORKDAY_HOURS = 9
DISTRIBUTORS = [f'DIST-{i:03d}' for i in range(1, 13)]
STAGE1_APPROVERS = ['Reviewer-A', 'Reviewer-B', 'Reviewer-C']
STAGE2_APPROVERS = ['Manager-X', 'Manager-Y']
STAGE1_MEAN_SERVICE = 6.0 # minutes
STAGE2_MEAN_SERVICE = 9.5 # minutes

def exp_sample(mean):
    return random.expovariate(1.0 / mean)

for day in range(DAYS):
    current_date = START_DATE + timedelta(days=day)
    if current_date.weekday() == 6: # skip Sunday
        continue
    is_month_end = current_date.day >= 26
    base_rate = 14 if is_month_end else 5 # invoices/hour
    # ... arrivals generated via Poisson process (exponential
    # inter-arrival times); each invoice assigned to the
    # earliest-free server at Stage 1, then (if it proceeds)
    # to the earliest-free server at Stage 2. Outcomes drawn
    # from weighted random choice (12% return, 7% reject).
    # Full listing: see simulate_data.py in the repository.
```

The complete, unabridged script — including the full arrival-generation loop, server-assignment logic, and CSV export — is available as `simulate_data.py` in the accompanying GitHub repository (Section 9), together with `analyze.py`, which implements Equations (1)–(6) and produces all figures in this report.

Appendix C: Glossary of Terms

Term	Meaning
Queue	A line of items (invoices) waiting for service.
Server	A resource that processes items, e.g. a reviewer or manager.
c	Number of parallel servers at a given stage.
λ (lambda)	Arrival rate — the mean number of invoices arriving per unit time.
μ (mu)	Service rate — the mean number of invoices one server can complete per unit time.
Offered load (a)	λ/μ — the amount of work arriving, measured in the same units as one server's capacity (Erlangs).
Utilisation (ρ)	Fraction of time servers are busy, a/c. Values approaching 1 indicate near-overload.
W_q	Mean time an item waits in queue before service begins.
W	Mean total time an item spends in the system (waiting + being served).
Little's Law	The relationship $L = \lambda W$, connecting queue length, arrival rate, and time in system.
Erlang-C formula	Gives the probability an arriving item must wait because all servers are busy.
M/M/c queue	A queue with Poisson (M) arrivals, exponential (M) service times, and c servers.
Tandem queue	A network where items pass through one queue and then another, in sequence.
Discrete-event simulation	A simulation that models a system as a sequence of individually timed events (arrivals, service completions).
Reneging	An item leaving a queue before being served, due to impatience — not modelled in this report.